

Lab 3: Architectural Justification — Remote Team Time-Tracking & Project Approver

Problem Statement #46 — Developer Tools & IT Operations

Architecture Selection

“We chose Microservices Architecture for the Remote Team Time-Tracking & Project Approver System.”

Architectural Choice

Microservices Architecture. The system is decomposed into independently deployable services — Time Tracking Service, Timesheet Approval Service, Budget Monitoring Service, Jira Integration Service, and Notification Service — fronted by a single API Gateway, rather than a single layered application or a monolithic client-server backend.

Two Reasons

- 1 **Independent scaling for uneven load:** The Budget Monitoring Service must refresh burn-rate dashboards within 500 ms under peak load (NFR-001), while the Time Tracking Service is invoked continuously by every remote developer throughout the day. Microservices allow each service to be scaled horizontally on its own (e.g., scaling Budget Monitoring during month-end reporting spikes) without over-provisioning the entire system, something a Layered or Client-Server design cannot do below the level of the whole application/server tier.
- 2 **Fault isolation around an unreliable external dependency:** Jira ticket synchronization (FR-004) depends on a third-party system whose uptime the team does not control. Isolating this integration inside its own Jira Integration Service means a Jira outage or API rate-limit only degrades ticket syncing — developers can still log time and submit timesheets, and managers can still approve them — whereas a tightly-coupled layered design risks a single failure cascading back through the shared business layer.

Security Advantage

All client traffic is funneled through a single API Gateway, which centralizes authentication and enforces role-based access control (NFR-002) before a request ever reaches the Timesheet Approval Service. For example, a developer's request to approve their own timesheet is rejected at the perimeter, and the team gets one place to audit, log, and rate-limit access instead of duplicating authorization logic inside every service.

Performance Benefit

Decoupling the Budget Monitoring Service from the Time Tracking Service means read-heavy dashboard queries (burn-rate calculations, live updates) can be cached and optimized independently, without contending for resources with the write-heavy path of continuous time-entry logging. This directly supports the sub-500ms dashboard refresh requirement (NFR-001) even under peak concurrent load, since the two workloads no longer share the same process or database connection pool.